Sesión 7

Programación en el Servidor y Objetos de Base de Datos

Carlos César Sánchez Coronel

BASES DE DATOS

Contenido general

1. Introducción
2. Vistas
3. Funciones y Procedimientos Almacenados
4. Triggers
5. Programación Avanzada
6. Funciones de Hash y Criptografía
7. Integración con Sistemas Externos
8. Comparativa entre Motores
9. Optimización de Código
10. Ejercicios Resueltos
11. Glosario

Contenido de la sección

1. Introducción
2. Vistas
3. Funciones y Procedimientos Almacenados
4. Triggers
5. Programación Avanzada
6. Funciones de Hash y Criptografía
7. Integración con Sistemas Externos
8. Comparativa entre Motores
9. Optimización de Código
10. Ejercicios Resueltos
11. Glosario

Objetivos de la sesión

- Comprender el papel de los objetos de base de datos: vistas, funciones, procedimientos, triggers.
- Aprender a crear y utilizar estos objetos para encapsular lógica de negocio.
- Conocer técnicas de hash y cifrado para datos sensibles.
- Integrar la base de datos con aplicaciones externas, ETL y modelos de IA.

Contenido de la sección

1. Introducción
2. Vistas
3. Funciones y Procedimientos Almacenados
4. Triggers
5. Programación Avanzada
6. Funciones de Hash y Criptografía
7. Integración con Sistemas Externos
8. Comparativa entre Motores
9. Optimización de Código
10. Ejercicios Resueltos
11. Glosario

Vistas Simples

Definición

Tabla virtual basada en una consulta. No almacena datos físicamente.

Ejemplo

```
CREATE VIEW clientes_activos AS
SELECT id, nombre, email
FROM clientes
WHERE activo = true;

SELECT * FROM clientes_activos WHERE ciudad = 'Madrid';
```

- **Seguridad**: ocultar columnas sensibles.
- **Simplicidad**: encapsular joins complejos.

Vistas Materializadas

Características

Almacenan físicamente el resultado de la consulta. Ideales para reportes.

PostgreSQL

```sql
CREATE MATERIALIZED VIEW resumen_ventas AS
SELECT producto_id, SUM(cantidad) AS total_vendido
FROM ventas GROUP BY producto_id;

REFRESH MATERIALIZED VIEW resumen_ventas;
REFRESH MATERIALIZED VIEW CONCURRENTLY resumen_ventas;
```

- **Ventaja**: consultas rápidas.
- **Desventaja**: datos no están en tiempo real.

Ejercicio Resuelto: Vista para Informes

Enunciado: Crear una vista que muestre para cada cliente su nombre, total gastado y número de pedidos (solo clientes con al menos un pedido).

Solución

```sql
CREATE VIEW resumen_clientes AS
SELECT c.id, c.nombre,
       COUNT(p.id) AS num_pedidos,
       COALESCE(SUM(p.total), 0) AS total_gastado
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id
GROUP BY c.id, c.nombre
HAVING COUNT(p.id) > 0;
```

Contenido de la sección

1. Introducción
2. Vistas
3. Funciones y Procedimientos Almacenados
4. Triggers
5. Programación Avanzada
6. Funciones de Hash y Criptografía
7. Integración con Sistemas Externos
8. Comparativa entre Motores
9. Optimización de Código
10. Ejercicios Resueltos
11. Glosario

Diferencias Clave

Función

- Retorna un valor (escalar o tabla).
- Se usa dentro de expresiones SQL.
- No puede realizar transacciones (commit/rollback) en la mayoría de motores.

Procedimiento

- Puede no retornar valor o devolver múltiples valores.
- Puede incluir transacciones.
- Se invoca con `CALL`.

Funciones Escalares

Ejemplo: Calcular IVA (PostgreSQL)

```
CREATE OR REPLACE FUNCTION calcular_iva(monto NUMERIC)
RETURNS NUMERIC AS $$
BEGIN
    RETURN COALESCE(monto, 0) * 1.21;
END;
$$ LANGUAGE plpgsql IMMUTABLE;

SELECT calcular_iva(100); -- 121
```

Ejemplo: MySQL

```
CREATE FUNCTION calcular_iva(monto DECIMAL(10,2))
RETURNS DECIMAL(10,2) DETERMINISTIC
BEGIN
    RETURN monto * 1.21;
END;
```

Funciones Tabla

Retornan un conjunto de filas

```
CREATE OR REPLACE FUNCTION empleados_por_depto(depto_id INT)
RETURNS TABLE(id INT, nombre TEXT, salario NUMERIC) AS $$
BEGIN
    RETURN QUERY
    SELECT id, nombre, salario
    FROM empleados
    WHERE departamento_id = depto_id;
END;
$$ LANGUAGE plpgsql STABLE;

SELECT * FROM empleados_por_depto(2);
```

Procedimientos Almacenados

Ejemplo: Transferencia bancaria (PostgreSQL)

```
CREATE OR REPLACE PROCEDURE transferir(
    origen INT, destino INT, monto NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE cuentas SET saldo = saldo - monto WHERE id = origen;
    UPDATE cuentas SET saldo = saldo + monto WHERE id = destino;
    COMMIT;
END;
$$;

CALL transferir(1, 2, 100);
```

Caso de Uso Real: Generación de Tickets en Gasolinera

Problema: Generar ticket correlativo por estación y fecha, con concurrencia.

Solución con función

```
CREATE TABLE ventas_gasolina (
    id SERIAL PRIMARY KEY,
    estacion_id INT NOT NULL,
    fecha DATE NOT NULL DEFAULT CURRENT_DATE,
    ticket_num INT NOT NULL,
    importe NUMERIC NOT NULL,
    UNIQUE (estacion_id, fecha, ticket_num)
);

CREATE OR REPLACE FUNCTION generar_ticket(p_estacion INT, p_importe NUMERIC)
RETURNS INT AS $$
DECLARE
    v_ticket INT;
BEGIN
    SELECT COALESCE(MAX(ticket_num), 0) + 1 INTO v_ticket
    FROM ventas_gasolina
    WHERE estacion_id = p_estacion AND fecha = CURRENT_DATE
    FOR UPDATE;

    INSERT INTO ventas_gasolina (estacion_id, fecha, ticket_num, importe)
    VALUES (p_estacion, CURRENT_DATE, v_ticket, p_importe);

    RETURN v_ticket;
```

Contenido de la sección

1. Introducción
2. Vistas
3. Funciones y Procedimientos Almacenados
4. Triggers
5. Programación Avanzada
6. Funciones de Hash y Criptografía
7. Integración con Sistemas Externos
8. Comparativa entre Motores
9. Optimización de Código
10. Ejercicios Resueltos
11. Glosario

Tipos de Triggers

BEFORE Se ejecuta antes de la operación. Útil para validaciones o modificar valores.

AFTER Después de la operación. Ideal para auditoría, sincronización.

INSTEAD OF Reemplaza la operación (en vistas).

FOR EACH ROW Se ejecuta por cada fila afectada.

FOR EACH STATEMENT Se ejecuta una vez por sentencia.

Trigger de Auditoría

Registrar cambios de salario

```sql
CREATE TABLE audit_salarios (
    id_empleado INT,
    salario_anterior NUMERIC,
    salario_nuevo NUMERIC,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    usuario TEXT DEFAULT current_user
);

CREATE OR REPLACE FUNCTION audit_salario_func()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.salario IS DISTINCT FROM NEW.salario THEN
        INSERT INTO audit_salarios (id_empleado, salario_anterior, salario_nuevo)
        VALUES (OLD.id, OLD.salario, NEW.salario);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trig_audit_salario
AFTER UPDATE OF salario ON empleados
FOR EACH ROW EXECUTE FUNCTION audit_salario_func();
```

Trigger de Validación

Evitar salarios negativos

```
CREATE OR REPLACE FUNCTION validar_salario()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.salario < 0 THEN
        RAISE EXCEPTION 'El salario no puede ser negativo';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trig_validar_salario
BEFORE INSERT OR UPDATE ON empleados
FOR EACH ROW EXECUTE FUNCTION validar_salario();
```

Precauciones con Triggers

- **Rendimiento**: overhead en cada operación DML, especialmente por fila.
- **Recursividad**: pueden dispararse a sí mismos; controlar con condiciones.
- **Depuración**: lógica oculta, difícil de rastrear.
- **Mantenibilidad**: efectos laterales no evidentes.

Contenido de la sección

1. Introducción
2. Vistas
3. Funciones y Procedimientos Almacenados
4. Triggers
5. Programación Avanzada
6. Funciones de Hash y Criptografía
7. Integración con Sistemas Externos
8. Comparativa entre Motores
9. Optimización de Código
10. Ejercicios Resueltos
11. Glosario

Cursores

Recorrer filas una por una (ineficiente, pero a veces necesario)

```
CREATE OR REPLACE FUNCTION procesar_empleados()
RETURNS VOID AS $$
DECLARE
    cur CURSOR FOR SELECT id, salario FROM empleados WHERE activo = true;
    v_id INT;
    v_salario NUMERIC;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_id, v_salario;
        EXIT WHEN NOT FOUND;
        UPDATE empleados SET salario = v_salario * 1.05 WHERE id = v_id;
    END LOOP;
    CLOSE cur;
END;
$$ LANGUAGE plpgsql;
```

Manejo de Excepciones

Capturar errores en procedimientos

```
CREATE OR REPLACE PROCEDURE transferir_seguro(
    origen INT, destino INT, monto NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE cuentas SET saldo = saldo - monto WHERE id = origen;
    UPDATE cuentas SET saldo = saldo + monto WHERE id = destino;
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE NOTICE 'Error en transferencia: %', SQLERRM;
END;
$$;
```

Contenido de la sección

1. Introducción
2. Vistas
3. Funciones y Procedimientos Almacenados
4. Triggers
5. Programación Avanzada
6. Funciones de Hash y Criptografía
7. Integración con Sistemas Externos
8. Comparativa entre Motores
9. Optimización de Código
10. Ejercicios Resueltos
11. Glosario

Conceptos Fundamentales

Hash Función unidireccional (no reversible). Ej: MD5, SHA-256.

Cifrado Reversible mediante clave (simétrico/asimétrico).

Salt Valor aleatorio añadido al hash para evitar ataques de diccionario.

Funciones Hash en SQL

PostgreSQL (requiere pgcrypto)

```sql
CREATE EXTENSION IF NOT EXISTS pgcrypto;
SELECT encode(digest('mi_secreto', 'sha256'), 'hex');
```

MySQL

```sql
SELECT MD5('mi_secreto');
SELECT SHA2('mi_secreto', 256);
```

SQL Server

```sql
SELECT HASHBYTES('SHA2_256', 'mi_secreto');
```

Almacenamiento Seguro de Contraseñas

Insertar usuario con hash + salt (PostgreSQL)

```
INSERT INTO usuarios (email, password_hash, salt)
VALUES (
    'usuario@mail.com',
    encode(digest('password123' || gen_salt('md5'), 'sha256'), 'hex'),
    gen_salt('md5')
);
```

Verificación

```
SELECT * FROM usuarios
WHERE email = 'usuario@mail.com'
AND password_hash = encode(digest('password_ingresada' || salt, 'sha256'), 'hex');
```

Anonimización de Datos

Sustituir emails por hash (entornos de test)

```
UPDATE clientes
SET email = encode(digest(email || 's3cr3t', 'sha256'), 'hex');
```

Precaución

Salt fijo no es seguro si el conjunto de emails es pequeño. Usar salt por registro.

Contenido de la sección

1. Introducción
2. Vistas
3. Funciones y Procedimientos Almacenados
4. Triggers
5. Programación Avanzada
6. Funciones de Hash y Criptografía
7. Integración con Sistemas Externos
8. Comparativa entre Motores
9. Optimización de Código
10. Ejercicios Resueltos
11. Glosario

¿Por qué integrar lógica en la base de datos?

- **Latencia**: código cerca de los datos.
- **Consistencia**: reglas centralizadas.
- **Seguridad**: control de acceso a nivel de fila/columna.
- **Rendimiento**: operaciones complejas optimizadas por el motor.

Llamada desde Python (psycopg2)

Ejemplo con función y procedimiento

```python
import psycopg2

conn = psycopg2.connect("dbname=test user=postgres")
cur = conn.cursor()

# Llamar a función que retorna valor
cur.callproc('generar_ticket', [5, 45.50])
ticket = cur.fetchone()[0]
print(f"Ticket: {ticket}")

# Llamar a procedimiento
cur.callproc('transferir', [1, 2, 100])
conn.commit()
cur.close()
conn.close()
```

Integración con Machine Learning

Función que expone datos para entrenamiento

```
CREATE OR REPLACE FUNCTION datos_entrenamiento(fecha_inicio DATE, fecha_fin DATE)
RETURNS TABLE(fecha DATE, producto_id INT, total_vendido NUMERIC) AS $$
BEGIN
    RETURN QUERY
    SELECT fecha, producto_id, total_vendido
    FROM resumen_diario
    WHERE fecha BETWEEN fecha_inicio AND fecha_fin;
END;
$$ LANGUAGE plpgsql STABLE;
```

Consumo en Python

```
import pandas as pd
df = pd.read_sql("SELECT * FROM datos_entrenamiento('2025-01-01', '2025-03-01')", conn)
```

Orquestación con Apache Airflow

DAG que ejecuta un procedimiento diario

```
from airflow import DAG
from airflow.providers.postgres.operators.postgres import PostgresOperator
from datetime import datetime

with DAG('etl_diario', start_date=datetime(2025,1,1), schedule='@daily') as dag:
    tarea = PostgresOperator(
        task_id='actualizar_resumen',
        postgres_conn_id='postgres_default',
        sql="CALL actualizar_resumen_diario();"
    )
```

Seguridad: SECURITY DEFINER vs INVOKER

SECURITY INVOKER (por defecto) Se ejecuta con privilegios del que llama.

SECURITY DEFINER Se ejecuta con privilegios del creador.

Ejemplo

```
REVOKE ALL ON usuarios FROM app_user; GRANT EXECUTE ON FUNCTION eliminar_usuario(INT) TO app_user;
```

Contenido de la sección

1. Introducción
2. Vistas
3. Funciones y Procedimientos Almacenados
4. Triggers
5. Programación Avanzada
6. Funciones de Hash y Criptografía
7. Integración con Sistemas Externos
8. Comparativa entre Motores
9. Optimización de Código
10. Ejercicios Resueltos
11. Glosario

Características de Programación

Característica	PostgreSQL	MySQL	SQL Server
Lenguaje procedural	PL/pgSQL, Python, etc.	SQL/PSM	T-SQL
Funciones escalares	Sí	Sí	Sí
Funciones tabla	Sí (RETURNS TABLE)	Sí (RETURNS)	Sí (TVF)
Procedimientos	Sí (CREATE PROCEDURE)	Sí	Sí
Triggers	BEFORE/AFTER	BEFORE/AFTER	INSTEAD OF, AFTER
Cursores	Sí	Sí	Sí
Vistas materializadas	Sí (REFRESH)	No nativas	Indexed views
SECURITY DEFINER	Sí	SQL SECURITY	EXECUTE AS

Contenido de la sección

1. Introducción
2. Vistas
3. Funciones y Procedimientos Almacenados
4. Triggers
5. Programación Avanzada
6. Funciones de Hash y Criptografía
7. Integración con Sistemas Externos
8. Comparativa entre Motores
9. Optimización de Código
10. Ejercicios Resueltos
11. Glosario

Buenas Prácticas

- Usar operaciones basadas en conjuntos en lugar de cursores.
- Evitar triggers innecesarios.
- Indexar columnas usadas en condiciones dentro de funciones.
- Marcar funciones como `IMMUTABLE`, `STABLE` o `VOLATILE` según corresponda.
- Analizar consultas con `EXPLAIN`.

Ejemplo de Optimización

Ineficiente (cursor)

```
FOR r IN SELECT salario FROM empleados WHERE departamento_id = p_depto LOOP
    total := total + r.salario;
END LOOP;
```

Eficiente (SQL directo)

```
SELECT SUM(salario) INTO total FROM empleados WHERE departamento_id = p_depto;
```

Contenido de la sección

1. Introducción
2. Vistas
3. Funciones y Procedimientos Almacenados
4. Triggers
5. Programación Avanzada
6. Funciones de Hash y Criptografía
7. Integración con Sistemas Externos
8. Comparativa entre Motores
9. Optimización de Código
10. Ejercicios Resueltos
11. Glosario

Ejercicio 1: Trigger de Stock

Enunciado: Al insertar en `detalle_pedido`, actualizar stock y abortar si insuficiente.

Solución

```
CREATE OR REPLACE FUNCTION actualizar_stock()
RETURNS TRIGGER AS $$
DECLARE
    v_stock INT;
BEGIN
    SELECT stock INTO v_stock FROM productos WHERE id = NEW.producto_id FOR UPDATE;
    IF v_stock < NEW.cantidad THEN
        RAISE EXCEPTION 'Stock insuficiente para producto %', NEW.producto_id;
    END IF;
    UPDATE productos SET stock = stock - NEW.cantidad WHERE id = NEW.producto_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trig_actualizar_stock
BEFORE INSERT ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION actualizar_stock();
```

Ejercicio 2: Función para Calcular Total de Pedido

Enunciado: Dado un `pedido_id`, calcular la suma de (cantidad * precio) y actualizar la columna `total` en `pedidos`.

Solución

```
CREATE OR REPLACE FUNCTION calcular_total_pedido(p_pedido_id INT)
RETURNS NUMERIC AS $$
DECLARE
    v_total NUMERIC;
BEGIN
    SELECT SUM(dp.cantidad * p.precio) INTO v_total
    FROM detalle_pedido dp
    JOIN productos p ON dp.producto_id = p.id
    WHERE dp.pedido_id = p_pedido_id;

    UPDATE pedidos SET total = v_total WHERE id = p_pedido_id;
    RETURN v_total;
END;
$$ LANGUAGE plpgsql;
```

Ejercicio 3: Vista con Datos Sensibles Ocultos

Enunciado: Crear vista que muestre nombre y salario, pero oculte salario a usuarios no autorizados.

Solución

```sql
CREATE VIEW empleados_rh AS
SELECT id, nombre,
       CASE WHEN CURRENT_USER = 'rh_user' THEN salario ELSE NULL END AS salario,
       departamento_id
FROM empleados;
```

Luego otorgar permisos select a los usuarios.

Ejercicio 4: Transferencia con Auditoría

Enunciado: Procedimiento que transfiera dinero entre cuentas y registre en auditoría.

Solución

```
CREATE TABLE auditoria_transferencias (
    id SERIAL PRIMARY KEY,
    origen INT, destino INT, monto NUMERIC,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    usuario TEXT DEFAULT current_user
);

CREATE OR REPLACE PROCEDURE transferir_con_auditoria(
    p_origen INT, p_destino INT, p_monto NUMERIC
)
LANGUAGE plpgsql AS $$
DECLARE
    v_saldo NUMERIC;
BEGIN
    SELECT saldo INTO v_saldo FROM cuentas WHERE id = p_origen FOR UPDATE;
    IF v_saldo < p_monto THEN
        RAISE EXCEPTION 'Saldo insuficiente';
    END IF;

    UPDATE cuentas SET saldo = saldo - p_monto WHERE id = p_origen;
    UPDATE cuentas SET saldo = saldo + p_monto WHERE id = p_destino;
    INSERT INTO auditoria_transferencias (origen, destino, monto)
    VALUES (p_origen, p_destino, p_monto);
```

Contenido de la sección

1. Introducción
2. Vistas
3. Funciones y Procedimientos Almacenados
4. Triggers
5. Programación Avanzada
6. Funciones de Hash y Criptografía
7. Integración con Sistemas Externos
8. Comparativa entre Motores
9. Optimización de Código
10. Ejercicios Resueltos
11. Glosario

Términos Clave

Vista Tabla virtual basada en una consulta.

Vista materializada Vista que almacena físicamente los datos.

Función escalar Retorna un valor simple.

Función tabla Retorna un conjunto de filas.

Procedimiento almacenado Bloque de código que realiza acciones.

Trigger Código que se ejecuta automáticamente ante un evento.

BEFORE / AFTER / INSTEAD OF Momento de ejecución del trigger.

FOR EACH ROW / STATEMENT Nivel del trigger.

Hash Función unidireccional (resumen).

Salt Valor aleatorio añadido al hash.

SECURITY DEFINER / INVOKER Modo de ejecución con permisos.

¡Gracias!